

Cálculo Paralelo de PI usando OpenMP

O número π pode ser aproximado por diferentes métodos numéricos. Um interessante método de aproximação é utilizar a integral:

$$\int_0^1 \frac{4}{1+x^2} dx = 4[\arctan(1) - \arctan(0)] = 4\left(\frac{\pi}{4} - 0\right) = \pi$$

Neste exercício, faça um programa que recebe o número de quadrados que essa integral irá ser dividida e retorne o valor aproximado de pi.

Por exemplo, considerando a divisão em $N = 2$ quadrados, a largura L de cada quadrado:

$$L = \frac{1}{2} = 0.5$$

Para a aproximação, é necessário usar os valores médios de x . O primeiro quadrado possui:

$$x_1 = (0 + 0,5) * L = 0,25$$

Aplicando a fórmula da integral acima, temos S_1 :

$$S_1 = \frac{4,0}{1,0 + x_1 * x_1} = \frac{4,0}{1,0 + 0,25 * 0,25} = 3,7647058824$$

O segundo quadrado possui:

$$x_2 = (1 + 0,5) * L = 0,75$$

$$S_2 = \frac{4,0}{1,0 + x_2 * x_2} = \frac{4,0}{1,0 + 0,75 * 0,75} = 2,56$$

Finalmente, temos o valor aproximado de pi, somando a área dos quadrados:

$$\pi \approx (S_1 + S_2) * L = (3,7647058824 + 2,56) * 0,5 = 6,3247058824 * 0,5 = 3,1623529412$$

Um valor muito mal aproximado (afinal, só usamos 2 termos). Neste exercício, iremos calcular o valor de pi com uma precisão melhor. Mas como essa integral converge lentamente, temos que usar o paralelismo. Você deverá desenvolver um programa utilizando OpenMP para calcular uma aproximação de π de forma paralela, distribuindo o trabalho entre múltiplos processos.

Entrada

A entrada possui o número N de retângulos que a região entre 0 a 1 será dividida ($2 \leq N \leq 10^{10}$).

Saída

A saída deve possuir o número de aproximado de PI, com 10 casas decimais.

Restrições

- A quantidade que o programa será dividido pode ser maior que 2 bilhões. Use inteiros de 64 bits.
- Para os valores com frações, utilize variáveis de precisão dupla.

Exemplo de Entrada 1

```
1235
```

Exemplo de Saída 1

```
3.1415927082
```

Exemplo de Entrada 2

```
10000000000
```

Exemplo de Saída 2

```
3.1415926536
```

É esperado que, com o uso de paralelismo, seu programa consiga executar em menos de 6 segundos

Author: Daniel Sundfeld daniel.sundfeld@unb.br